# Mittens 확장 개념 설명

## 개요

Mittens는 2018년 Nick Dingwall과 Christopher Potts가 개발한 GloVe의 확장 알고리즘입니다. Mittens는 "도메인 특화 표현 학습을 위한 GloVe 확장"이라는 부제로, 기존 GloVe 알고리즘에 **정규화 항(regularization term)**을 추가하여 사전 훈련된 임베딩을 활용한 도메인 적응을 가능하게 합니다.

**용어 통일**: 이 문서는 GloVe 관련 문서들과 동일한 용어를 사용합니다. “임베딩”, “동시출현”, “벡터 표현” 등이 일관되게 사용되며, 새로운 개념인 “정규화 항”, “warm start”, “도메인 적응” 등의 용어도 명확히 정의됩니다.

## 도메인 적응 문제의 동기

### 기존 GloVe의 한계

일반적인 GloVe 학습 과정에서는 다음과 같은 문제들이 발생할 수 있습니다:

1. **데이터 부족**: 특정 도메인의 코퍼스가 작을 경우 품질 좋은 임베딩 학습 어려움
2. **도메인 특화성**: 일반 도메인에서 학습된 임베딩이 특수 도메인에서 부적절할 수 있음
3. **처음부터 학습**: 기존의 풍부한 언어 지식을 활용하지 못하고 무작위 초기화부터 시작

### 전이 학습의 필요성

현실적으로 많은 자연어 처리 응용에서는:

- **대규모 일반 코퍼스**에서 학습된 고품질 임베딩이 이미 존재
- **특정 도메인 코퍼스**는 상대적으로 작지만 도메인 특화 정보 포함
- 두 정보를 효과적으로 결합할 방법이 필요

### Mittens의 해결 접근법

Mittens는 이 문제를 **정규화된 목적 함수**를 통해 해결합니다:

```
J_mittens = J_glove + λ × Regularization_term
```

여기서 정규화 항은 학습된 임베딩이 사전 훈련된 임베딩으로부터 너무 멀어지지 않도록 제약을 가합니다.

## Mittens의 확장된 목적 함수

### 기본 GloVe 목적 함수 복습

GloVe 이론 문서에서 유도된 기존 GloVe의 목적 함수는 다음과 같습니다:

```
J_glove = Σ_{i,j=1}^V f(X_ij)(w_i^T w_j + b_i + b_j - log X_ij)²
```

### Mittens 정규화 항 추가

Mittens는 여기에 정규화 항을 추가합니다:

```
J_mittens = J_glove + mittens × Σ_{i ∈ S} ||G_i - G_i^(0)||²
```

이 확장된 목적 함수는 기본 GloVe 이론을 기반으로 하며, 실제 구현에서 어떻게 코드로 구현되는지 확인할 수 있습니다.

**수학적 구성 요소**:

- **G_i = W_i + C_i**: 단어 i의 최종 임베딩 (단어 벡터 + 문맥 벡터)
- **G_i^(0)**: 단어 i의 사전 훈련된 임베딩
- **S**: 사전 훈련된 임베딩이 있는 단어들의 집합

- **mittens**: 정규화 강도를 조절하는 하이퍼파라미터 (λ)

### 정규화 항의 의미

정규화 항 `||G_i - G_i^(0)||²` 는 다음을 의미합니다:

1. **거리 제약**: 학습된 임베딩이 원본 임베딩으로부터 유클리드 거리상 너무 멀어지지 않도록 제약
2. **선택적 적용**: 사전 훈련된 임베딩이 있는 단어에만 적용 (집합 S)
3. **유연성**: mittens 매개변수로 제약 강도 조절 가능

## Warm Start 초기화 전략 (사전 지식 활용 초기화)

### 기존 GloVe의 무작위 초기화

일반적인 GloVe에서는 모든 매개변수를 Xavier Glorot 방법으로 무작위 초기화합니다:

```
self.W = randmatrix(n_words, self.n) # 무작위 초기화
self.C = randmatrix(n_words, self.n) # 무작위 초기화
```

### Mittens의 Warm Start 전략

Mittens는 사전 훈련된 임베딩이 있는 단어들에 대해 **warm start**(사전 지식 활용 초기화)를 수행합니다:

```
if w in initial_embedding_dict:
self.has_embedding[i] = 1  # 사전 임베딩 존재 표시
embedding = np.array(initial_embedding_dict[w])
self.original_embedding[i] = embedding # 원본 임베딩 저장
# 대칭성 깨뜨리기 위한 노이즈 추가
```

```
self.W[i] = 0.5 * embedding + noise(self.n)

self.C[i] = 0.5 * embedding + noise(self.n)
```

### Warm Start의 핵심 아이디어

1. **분할 초기화**: 사전 임베딩을 W와 C에 절반씩 할당

```
W_i^(0) = 0.5 × G_i^(0) + ϵ_w

C_i^(0) = 0.5 × G_i^(0) + ϵ_c
```

2. **대칭성 파괴**: 작은 노이즈 ε를 추가하여 W와 C가 동일한 그래디언트를 받지 않도록 함
3. **일관성 유지**: W_i^(0) + C_i^(0) ≈ G_i^(0)로 초기 상태에서 원본 임베딩과 유사

### 노이즈 추가의 필요성

```
def  noise(n, scale=0.01):

return np.random.normal(0, scale, size=n)
```

**수학적 근거**:

- W_i = C_i인 경우, 두 매개변수의 그래디언트가 동일해짐
- 이로 인해 학습 과정에서 W와 C가 계속 동일하게 유지됨
- 작은 노이즈로 대칭성을 깨뜨려 독립적인 학습 가능

## 하이퍼파라미터 mittens의 역할

### mittens 매개변수의 의미

`mittens` 매개변수는 정규화 강도를 조절하는 핵심 하이퍼파라미터입니다:

```
def __init__(self, n=100, mittens=0.1, ...):
self.mittens = float(mittens)
```

### mittens 값에 따른 동작

1. **mittens = 0**: 순수한 GloVe (정규화 없음)

```
J = J_glove
```

2. **mittens > 0**: Mittens 정규화 적용

```
J = J_glove + mittens × Regularization
```

3. **mittens → ∞**: 사전 임베딩 고정 (학습 없음)

### 적절한 mittens 값 선택

**일반적인 권장값**:

- **mittens = 0.1**: 기본값, 대부분의 경우에 적절
- **mittens = 0.01**: 약한 정규화, 도메인 적응을 크게 허용
- **mittens = 1.0**: 강한 정규화, 사전 임베딩에 가깝게 유지

**선택 기준**:

- **도메인 차이가 클 때**: 작은 mittens 값 (0.01-0.05)
- **도메인 차이가 작을 때**: 큰 mittens 값 (0.5-1.0)
- **데이터가 매우 적을 때**: 큰 mittens 값으로 과적합 방지

# 그래디언트 계산에서의 정규화 항 적용

## 기존 GloVe 그래디언트

```
# GloVe 그래디언트 계산

diffs = predictions - log_coincidence

weighted_diffs = np.multiply(weights, diffs)

wgrad = weighted_diffs.dot(self.C)

cgrad = weighted_diffs.T.dot(self.W)

error = (0.5 * np.multiply(weights, diffs ** 2)).sum()
```

## Mittens 정규화 항 추가

```
# Mittens 정규화 항 추가 (mittens > 0인 경우에만)

if  self.mittens > 0:

curr_embedding = self.W + self.C # 현재 임베딩

distance = curr_embedding[self.has_embedding, :] - \

self.original_embedding[self.has_embedding, :]

# 그래디언트에 정규화 항 추가

wgrad[self.has_embedding, :] += 2 * self.mittens * distance

cgrad[self.has_embedding, :] += 2 * self.mittens * distance

# 손실에 정규화 항 추가

error += self.mittens * (np.linalg.norm(distance, ord=2, axis=1) ** 2).sum()
```

### 정규화 그래디언트의 수학적 유도

정규화 항에 대한 그래디언트:

```
∂/∂W_i [mittens × ||G_i - G_i^(0)||²] = 2 × mittens × (G_i - G_i^(0))

∂/∂C_i [mittens × ||G_i - G_i^(0)||²] = 2 × mittens × (G_i - G_i^(0))
```

여기서 G_i = W_i + C_i이므로:

```
∂G_i/∂W_i = 1, ∂G_i/∂C_i = 1
```

따라서 W와 C 모두에 동일한 정규화 그래디언트가 추가됩니다.

### 선택적 정규화 적용

```
distance = curr_embedding[self.has_embedding, :] - \

self.original_embedding[self.has_embedding, :]
```

**핵심 특징**:

- `self.has_embedding` : 사전 임베딩이 있는 단어들의 불린 마스크
- 사전 임베딩이 없는 단어들은 순수 GloVe 학습
- 사전 임베딩이 있는 단어들만 정규화 제약 적용

## 실용적 활용 사례

### 1. 도메인 특화 임베딩 학습

**시나리오**: 의료 도메인 텍스트 분석

- **일반 임베딩**: Wikipedia, Common Crawl 등에서 학습된 GloVe
- **도메인 코퍼스**: 의료 논문, 병원 기록 등
- **목표**: 일반 언어 지식 + 의료 전문 용어 이해

**Mittens 적용**:

```
# 일반 도메인에서 사전 훈련된 임베딩 로드 (아래 구현 참조)
general_embeddings = load_glove_embeddings('glove.6B.300d.txt')

# 의료 도메인 동시출현 행렬 구축 (사용자 구현 필요)
# 예: sklearn.feature_extraction.text.CountVectorizer 등을 활용
medical_cooccurrence = build_cooccurrence_matrix(medical_corpus)

# Mittens로 도메인 적응
mittens = Mittens(n=300, mittens=0.1)
domain_embeddings = mittens.fit(
medical_cooccurrence,
vocab=medical_vocab,
initial_embedding_dict=general_embeddings
)
```

## 2. 다국어 임베딩 정렬

**시나리오**: 영어 임베딩을 한국어 도메인에 적응

- **사전 임베딩**: 영어 GloVe 임베딩

- **대상 언어**: 한국어 코퍼스
- **정렬 방법**: 이중언어 사전을 통한 초기 매핑

**활용 예시**:

```
# 영어-한국어 번역 사전을 통한 초기 임베딩 구축
en_ko_dict = load_bilingual_dictionary()
initial_ko_embeddings = {}

for ko_word, en_word in en_ko_dict.items():
if en_word in english_embeddings:
initial_ko_embeddings[ko_word] = english_embeddings[en_word]

# 한국어 코퍼스로 Mittens 학습
mittens = Mittens(n=300, mittens=0.2)
ko_embeddings = mittens.fit(
korean_cooccurrence,
vocab=korean_vocab,
initial_embedding_dict=initial_ko_embeddings
)
```

## 3. 시간적 언어 변화 추적

**시나리오**: 언어의 시간적 변화 분석

- **기준 임베딩**: 과거 시점의 임베딩

- **새로운 데이터**: 최근 시점의 코퍼스
- **분석 목표**: 단어 의미의 변화 정도 측정

**분석 방법**:

```
# 2010년 임베딩을 기준으로 2020년 데이터 학습
mittens_weak = Mittens(mittens=0.01) # 약한 정규화
embeddings_2020 = mittens_weak.fit(
cooccurrence_2020,
initial_embedding_dict=embeddings_2010
)

# 의미 변화 정도 측정
semantic_shifts = compute_embedding_distances(
embeddings_2010, embeddings_2020
)
```

# 사전 훈련된 임베딩 활용법

## 1. 임베딩 형식과 로딩

**일반적인 GloVe 형식**:

```
word1 0.1 0.2 0.3 ... 0.n
word2 0.4 0.5 0.6 ... 0.m
```

```
...
```

**Python 로딩 코드:**

```
def load_glove_embeddings(filepath):
embeddings = {}
with open(filepath, 'r', encoding='utf-8') as f:
for line in f:
parts = line.strip().split()
word = parts[0]
vector = np.array([float(x) for x in parts[1:]])
embeddings[word] = vector
return embeddings
```

## 2. 어휘 정렬과 차원 일치

**어휘 교집합 처리:**

```
def align_vocabularies(target_vocab, pretrained_embeddings):
aligned_embeddings = {}
missing_words = []
for word in target_vocab:
if word in pretrained_embeddings:
aligned_embeddings[word] = pretrained_embeddings[word]
else:
```

```
missing_words.append(word)
print(f"Found embeddings for {len(aligned_embeddings)} words")
print(f"Missing embeddings for {len(missing_words)} words")
return aligned_embeddings, missing_words
```

**차원 일치 확인**:

```
def validate_embedding_dimensions(embeddings, expected_dim):
for word, vector in embeddings.items():
if len(vector) != expected_dim:
raise ValueError(f"Dimension mismatch for word '{word}': "
f"expected {expected_dim}, got {len(vector)}")
```

## 3. 품질 평가와 전처리

**임베딩 품질 확인**:

```
def evaluate_embedding_quality(embeddings):
# 벡터 크기 분포 확인
norms = [np.linalg.norm(vec) for vec in embeddings.values()]
print(f"Vector norm statistics:")
print(f" Mean: {np.mean(norms):.3f}")
print(f" Std: {np.std(norms):.3f}")
# 유사도 분포 확인
words = list(embeddings.keys())[:1000] # 샘플링
```

```
similarities = []

for i in  range(len(words)):

for j in  range(i+1, len(words)):

# 코사인 유사도 계산 (numpy 사용)

sim = np.dot(embeddings[words[i]], embeddings[words[j]]) / (

np.linalg.norm(embeddings[words[i]]) * np.linalg.norm(embeddings[words[j]]))

similarities.append(sim)

print(f"Cosine similarity statistics:")

print(f" Mean: {np.mean(similarities):.3f}")

print(f" Std: {np.std(similarities):.3f}")
```

## 매개변수 설정 가이드라인

### 1. mittens 매개변수 튜닝

**그리드 서치 예시:**

```
mittens_values = [0.01, 0.05, 0.1, 0.2, 0.5, 1.0]

best_score = -float('inf')

best_mittens = None


for mittens_val in mittens_values:

model = Mittens(mittens=mittens_val, max_iter=100)

embeddings = model.fit(X, vocab=vocab,
```

```
initial_embedding_dict=pretrained)

# 평가 (예: 단어 유사도 태스크)

score = evaluate_word_similarity(embeddings, test_set)

if score > best_score:

best_score = score

best_mittens = mittens_val



print(f"Best mittens value: {best_mittens}")
```

## 2. 학습률과 반복 횟수 조정

**Mittens에서의 권장 설정**:

```
# 일반적인 설정

mittens = Mittens(

n=300, # 임베딩 차원

mittens=0.1, # 정규화 강도

max_iter=1000, # 최대 반복 횟수

learning_rate=0.05, # 학습률

tol=1e-4, # 수렴 허용 오차

xmax=100, # 가중치 함수 포화점

alpha=0.75  # 가중치 함수 지수

)
```

**도메인별 조정**:

- **큰 도메인 차이**: learning_rate=0.01, mittens=0.05
- **작은 도메인 차이**: learning_rate=0.1, mittens=0.5
- **매우 작은 데이터**: max_iter=500, mittens=1.0

### 3. 수렴 모니터링

```
# 학습 곡선 시각화
import matplotlib.pyplot as plt

mittens = Mittens(mittens=0.1, display_progress=10)
embeddings = mittens.fit(X, vocab=vocab,
initial_embedding_dict=pretrained)

# 손실 곡선 플롯
plt.figure(figsize=(10, 6))
plt.plot(mittens.errors)
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('Mittens Training Loss')
plt.yscale('log')
plt.grid(True)
plt.show()
```

# Mittens vs GloVe 비교 분석

## 성능 비교

| 측면 | GloVe | Mittens |

|------|-------|---------|

| **학습 데이터** | 대용량 필요 | 소용량도 가능 |

| **도메인 적응** | 어려움 | 자연스러움 |

| **사전 지식 활용** | 불가능 | 가능 |

| **계산 복잡도** | $O(V^2 \times n)$ | $O(V^2 \times n)$ + 정규화 |

| **메모리 사용량** | 기본 | +사전 임베딩 저장 |

## 사용 시나리오별 권장

**GloVe 권장**:

- 충분한 도메인 특화 데이터 보유
- 처음부터 새로운 임베딩 학습 필요
- 계산 자원이 제한적인 경우

**Mittens 권장**:

- 제한된 도메인 데이터
- 기존 고품질 임베딩 활용 가능
- 도메인 적응이 주요 목표

# 결론

Mittens는 GloVe의 강력한 확장으로, 다음과 같은 핵심 기여를 제공합니다:

### 주요 혁신

1. **정규화 기반 도메인 적응**: 사전 임베딩을 제약으로 활용
2. **Warm Start 전략**: 사전 지식을 활용한 효율적 초기화로 빠른 수렴
3. **선택적 정규화**: 사전 임베딩이 있는 단어에만 제약 적용
4. **유연한 제어**: mittens 매개변수로 적응 강도 조절

### 실용적 가치

- **데이터 효율성**: 작은 도메인 코퍼스로도 고품질 임베딩 학습
- **지식 전이**: 대규모 일반 도메인 지식을 특수 도메인에 전이
- **점진적 학습**: 기존 임베딩을 점진적으로 개선
- **다양한 응용**: 도메인 적응, 다국어 정렬, 시간적 변화 추적 등

Mittens는 현실적인 자연어 처리 환경에서 자주 마주치는 "제한된 도메인 데이터"와 “기존 지식 활용” 문제에 대한 우아한 해결책을 제시합니다. 이를 통해 연구자와 개발자들은 더 효율적이고 실용적인 단어 임베딩을 구축할 수 있게 되었습니다.

이러한 Mittens의 개념이 실제로 어떻게 구현되는지는 구현체 분석에서, 성능 특성은 성능 최적화에서 확인할 수 있습니다.

## 관련 문서

Mittens 확장을 완전히 이해하기 위해 다음 문서들을 참조하세요:

- **GloVe 이론적 배경**: Mittens가 확장하는 기본 GloVe 알고리즘의 이론
- **GloVe 알고리즘 분석**: Mittens가 기반으로 하는 GloVe 학습 과정
- **구현체 코드 분석**: Mittens 알고리즘의 실제 구현 방법
- **성능 최적화 분석**: Mittens 구현의 성능 특성과 최적화

**학습 흐름**: GloVe 기초(01,02) → **Mittens 확장(03)** → 구현(04) → 성능(05)